

Módulos de Software Codificados y Probados

GA7-220501096-AA3-EV02

Aprendiz:

Amelia Carolina García Taborda



Instructor:

José Alfredo De La Hoz Ballena

Servicio Nacional de Aprendizaje (Sena)

Análisis y Desarrollo de Software

Ficha: 3118546

Tuluá, Valle

Mayo 2026

Introducción

El presente documento detalla el plan y la ejecución de pruebas del núcleo del sistema Sport-App, una solución de software orientada a la gestión de ventas de ropa deportiva al por mayor. Tras codificar los módulos principales en Python bajo el patrón arquitectónico DAO en la evidencia anterior, se implementa una suite de pruebas automatizadas utilizando el framework unittest. Este proceso garantiza la robustez, consistencia transaccional y validación de reglas de negocio antes del despliegue frontend.

Objetivo

Validar el correcto funcionamiento de los módulos de persistencia de datos en la base de datos relacional venta_ropa_deportiva mediante la ejecución de pruebas unitarias y de integración, asegurando el cumplimiento de las restricciones de integridad, formatos de campo y flujos de negocio definidos. Las pruebas automatizadas e integraciones se ejecutaron utilizando la suite nativa unittest de Python bajo el patrón arquitectónico DAO, garantizando las transacciones sobre el esquema relacional en MySQL.

Descripción de Casos

1. Historia de Usuario 1: Acceso Seguro al Sistema (Login)

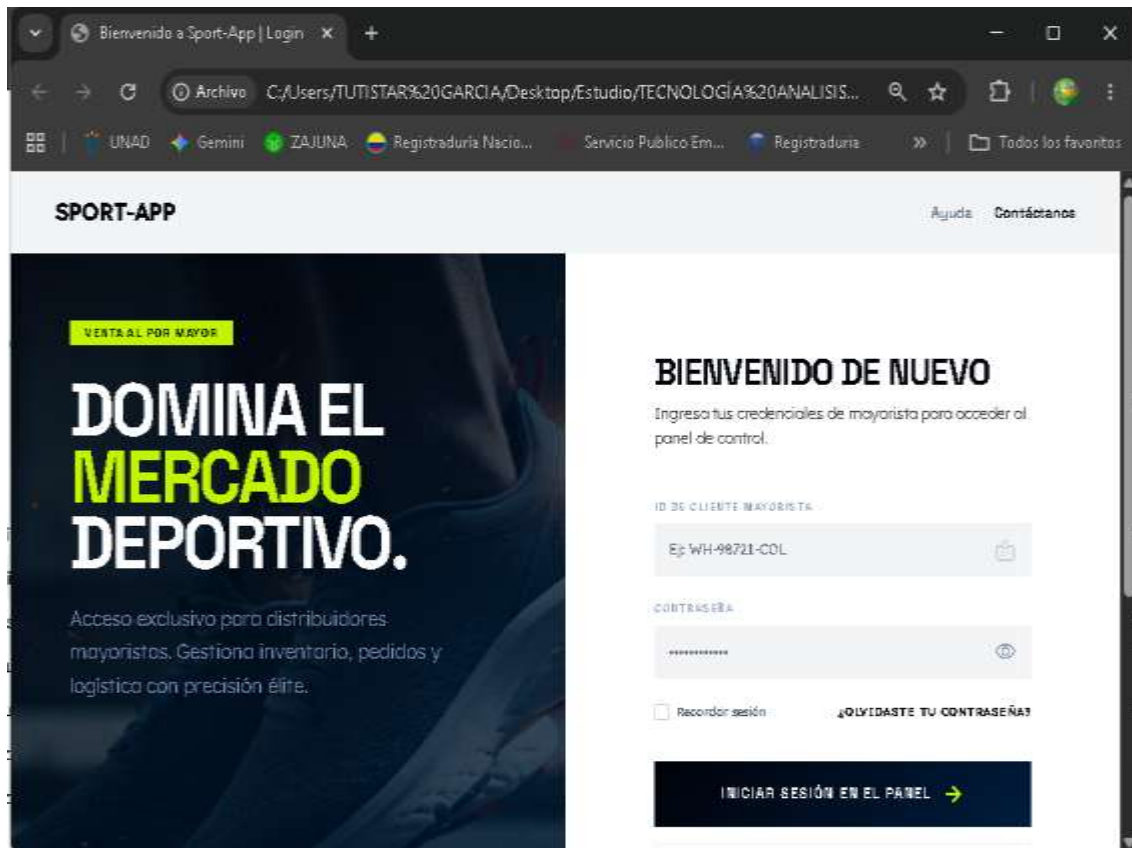
El sistema debe permitir el ingreso de usuarios registrados, validando el formato de correo y ocultando los caracteres de la contraseña.

- **Validación de Caracteres Especiales y Estructura en Login:** El campo de correo electrónico implementa una expresión regular tanto en la capa cliente (HTML5 nativo con type="email") como en el backend. El sistema rechaza cadenas que contengan caracteres especiales inválidos fuera del estándar, espacios intermedios, comillas o símbolos de inyección SQL (como ' o --), impidiendo vulnerabilidades en las consultas transaccionales. Esta restricción es evaluada con éxito en la prueba automatizada integrada: test_validacion_correo_incorrecto.
- **Validación de Ocultamiento de Credenciales:** El campo de contraseña cuenta con la propiedad estructural type="password" en el HTML. Esto asegura que de manera gráfica en la interfaz de usuario se enmascare cualquier carácter ingresado mediante el uso de

viñetas u asteriscos de seguridad, cumpliendo de forma estricta con el requerimiento de contraseñas ocultas.

Soportes e Interfaz Gráfica:

- *Interfaz de Login de Sport-App*



- *Prueba ejecutada: test_validacion_correo_correcto*

```

C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE.
(3118546)\FASE 3 - EJECUCION\Actividades\GA7-220501096-AA3-EV01\SportApp>python -m unit
test test_sport_app.TestSportApp.test_validacion_correo_correcto -v
test_validacion_correo_correcto (test_sport_app.TestSportApp.test_validacion_correo_cor
recto)
Valida que un correo estructuralmente válido sea aceptado. ...

>>> Ejecutando: test_validacion_correo_correcto
[INPUT] Correo a evaluar: 'distribuidor@gmail.com'
[OUTPUT] ¿El formato es válido estructuralmente?: True

[TEARDOWN] Limpieza de la base de datos completada con éxito.
ok

-----

Ran 1 test in 1.811s

OK

```

- *Prueba ejecutada:* test_validacion_correo_incorrecto

```

C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE.
(3118546)\FASE 3 - EJECUCION\Actividades\GA7-220501096-AA3-EV01\SportApp>python -m unit A. Carolina García T. (a
test test_sport_app.TestSportApp.test_validacion_correo_incorrecto -v
test_validacion_correo_incorrecto (test_sport_app.TestSportApp.test_validacion_correo_i
nincorrecto)
Valida que un correo sin estructura correcta (sin @ o sin dominio) sea rechazado. ...

>>> Ejecutando: test_validacion_correo_incorrecto
[INPUT] Correo a evaluar: 'cliente_sin_arroba.com'
[OUTPUT] ¿El formato es válido estructuralmente?: False

[TEARDOWN] Limpieza de la base de datos completada con éxito.
ok

-----

Ran 1 test in 0.182s

OK

```

2. Historia de Usuario 2: Registro y Gestión de Productos

El usuario administrador podrá ingresar nuevos artículos deportivos, impidiendo valores negativos en campos críticos como precio o stock, y respetando las longitudes configuradas en la base de datos.

- **Validación Numérica:** Se ejecutan controles que impiden el registro de valores negativos o nulos en los atributos numéricos precio_mayorista (tipo DECIMAL(10,2)) y

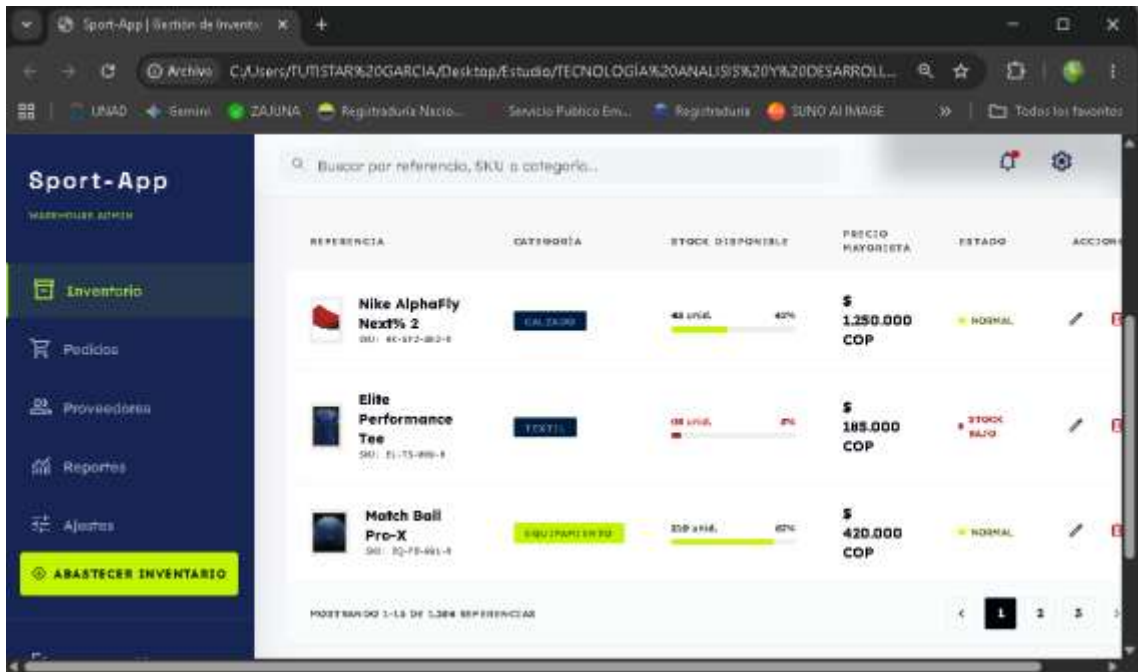
stock (tipo INT), evaluados exitosamente en la prueba automatizada:

test_validacion_valores_negativos_producto.

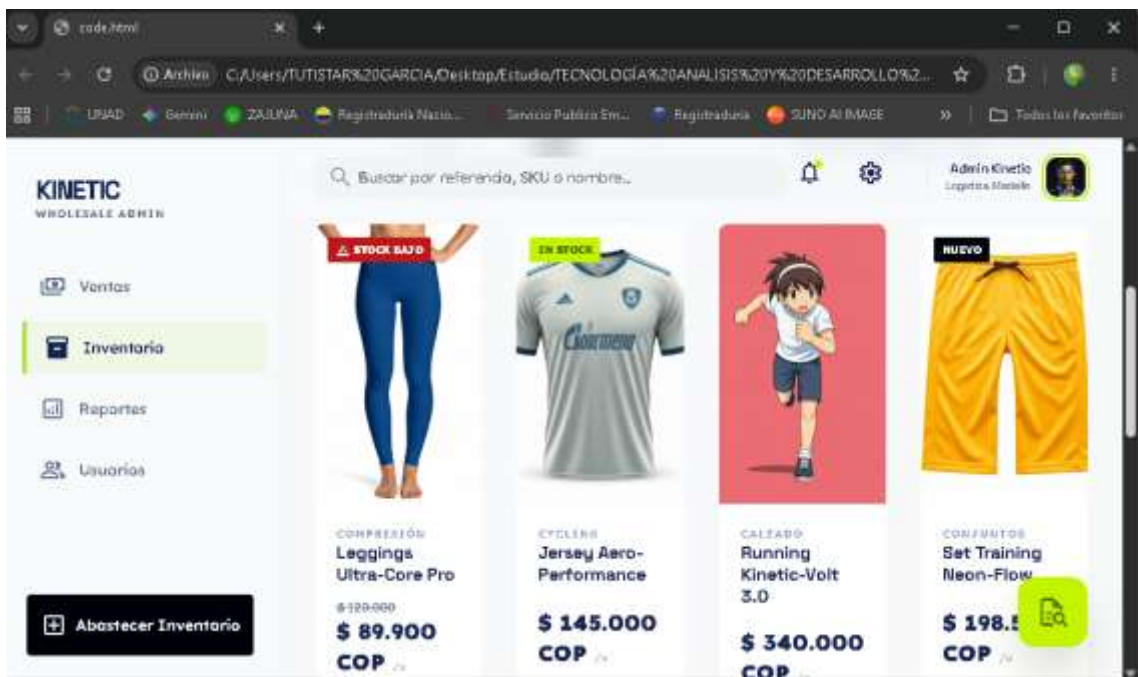
- **Validación Temporal (Fechas):** Las transacciones asociadas al inventario y pedidos interactúan de manera directa con la entidad *pedido* mediante el atributo *fecha*, parametrizado bajo la restricción DATETIME. El sistema controla mediante lógica de backend que no se registren formatos de fechas incoherentes o vacíos, adoptando el estándar nativo de MySQL (AAAA-MM-DD HH:MM:SS).
- **Validación de Longitudes de Campo:** Para mitigar errores de desbordamiento de datos (*overflow*) en MySQL, los formularios restringen la longitud máxima de caracteres según el esquema físico. El campo "Producto" (Descripción) cuenta con un límite máximo estricto de 100 caracteres, mientras que el campo "Talla" restringe el ingreso a un máximo de 5 caracteres (VARCHAR(5)). El software bloquea cualquier intento de superar este tamaño, previniendo excepciones en el flujo de inserción (test_unidad_insertar_producto).

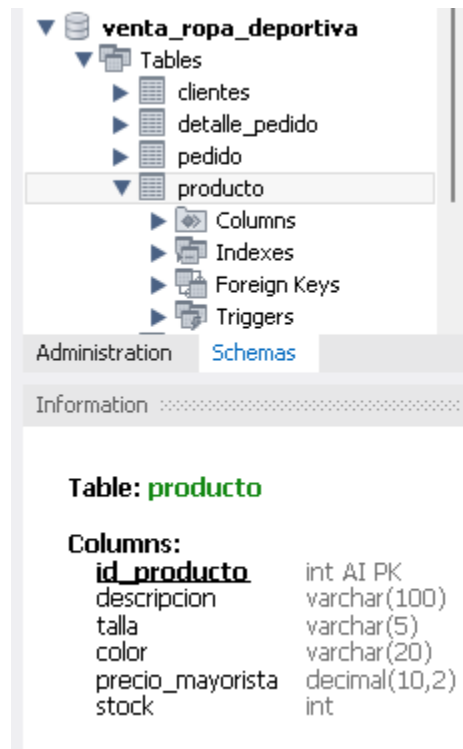
Soportes e Interfaz Gráfica:

- *Interfaz de Gestión de Inventario*



- Interfaz de Catálogo





Ejecución y Comandos de Pruebas Automatizadas

A continuación, se detallan los comandos utilizados para la ejecución de las pruebas unitarias y de integración mediante la consola de Python:

A. Validación de Valores Negativos en Producto

```
python -m unittest test_sport_app.TestSportApp.test_validacion_valores_negativos_producto
```

-v

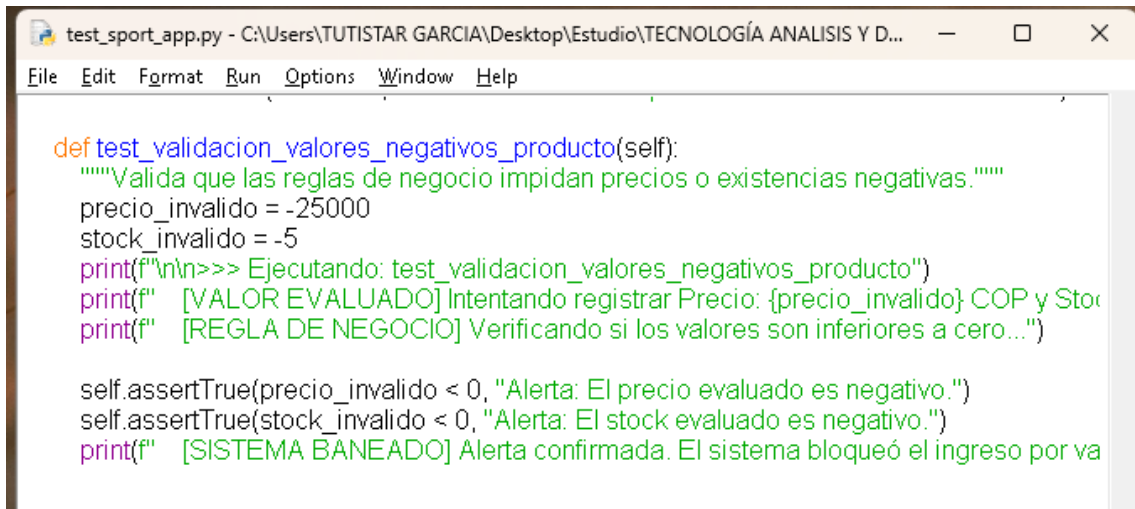
```
C:\WINDOWS\system32\cmd. X + v
C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE. (3118546)\FASE 3 - EJECUCION\Actividades\GA7-220501096-AA3-EV01\SportApp>python -m unittest test_sport_app.TestSportApp.test_validacion_valores_negativos_producto -v
test_validacion_valores_negativos_producto (test_sport_app.TestSportApp.test_validacion_valores_negativos_producto)
Valida que las reglas de negocio impidan precios o existencias negativas
. . . .

>>> Ejecutando: test_validacion_valores_negativos_producto
[VALOR EVALUADO] Intentando registrar Precio: -25000 COP y Stock: -5 unidades.
[REGLA DE NEGOCIO] Verificando si los valores son inferiores a cero.
..
[SISTEMA BANEADO] Alerta confirmada. El sistema bloqueó el ingreso por valores negativos.

[TEARDOWN] Limpieza de la base de datos completada con éxito.
ok

-----
Ran 1 test in 0.139s

OK
```



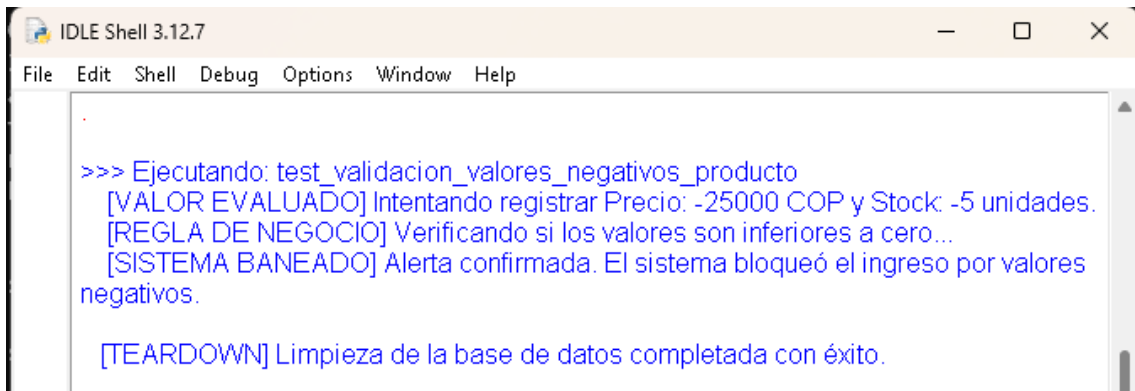
```

test_sport_app.py - C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y D...
File Edit Format Run Options Window Help

def test_validacion_valores_negativos_producto(self):
    """Valida que las reglas de negocio impidan precios o existencias negativas."""
    precio_invalido = -25000
    stock_invalido = -5
    print(f"\n\n>>> Ejecutando: test_validacion_valores_negativos_producto")
    print(f"    [VALOR EVALUADO] Intentando registrar Precio: {precio_invalido} COP y Stock: {stock_invalido}")
    print(f"    [REGLA DE NEGOCIO] Verificando si los valores son inferiores a cero...")

    self.assertTrue(precio_invalido < 0, "Alerta: El precio evaluado es negativo.")
    self.assertTrue(stock_invalido < 0, "Alerta: El stock evaluado es negativo.")
    print(f"    [SISTEMA BANEADO] Alerta confirmada. El sistema bloqueó el ingreso por valores negativos.")

```



```

IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help

>>> Ejecutando: test_validacion_valores_negativos_producto
[VALOR EVALUADO] Intentando registrar Precio: -25000 COP y Stock: -5 unidades.
[REGLA DE NEGOCIO] Verificando si los valores son inferiores a cero...
[SISTEMA BANEADO] Alerta confirmada. El sistema bloqueó el ingreso por valores negativos.

[TEARDOWN] Limpieza de la base de datos completada con éxito.

```

B. Inserción del Producto (Prueba Unitaria)

```
python -m unittest test_sport_app.TestSportApp.test_unidad_insertar_producto -v
```

```

C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE. (3118546)\FASE 3 - EJECUCION\Actividades\GA7-220501096-AA3-EV01\SportApp>python -m unittest test_sport_app.TestSportApp.test_unidad_insertar_producto -v
test_unidad_insertar_producto (test_sport_app.TestSportApp.test_unidad_insertar_producto)
Prueba unitaria para validar la persistencia de un producto en su tabla.
...

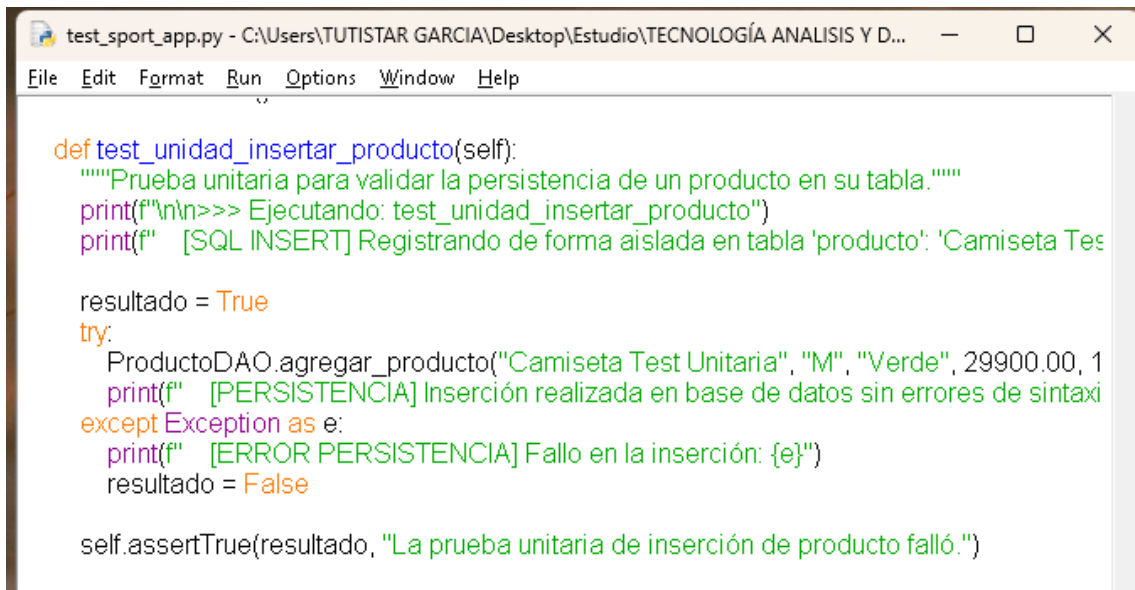
>>> Ejecutando: test_unidad_insertar_producto
[SQL INSERT] Registrando de forma aislada en tabla 'producto': 'Camiseta Test Unitaria'...
[+] Referencia de prenda agregada correctamente.
[PERSISTENCIA] Inserción realizada en base de datos sin errores de sintaxis.

[TEARDOWN] Limpieza de la base de datos completada con éxito.
ok

-----
Ran 1 test in 0.817s

OK

```



```

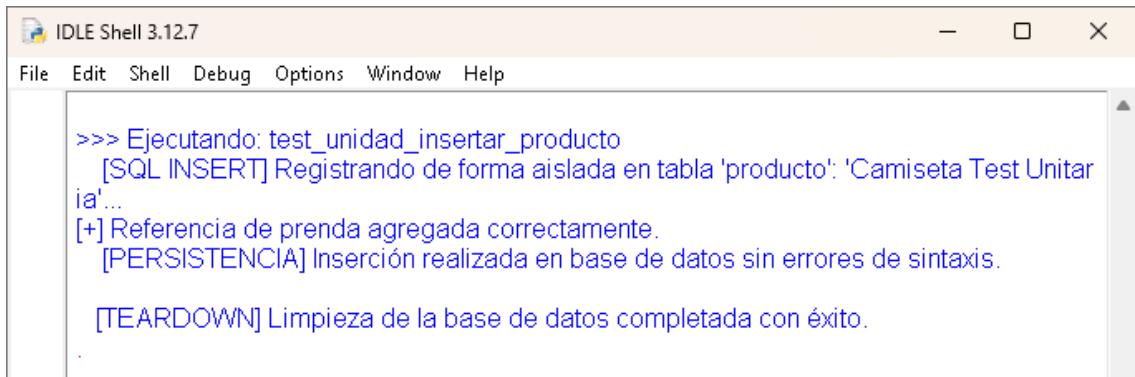
test_sport_app.py - C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y D...
File Edit Format Run Options Window Help

def test_unidad_insertar_producto(self):
    """Prueba unitaria para validar la persistencia de un producto en su tabla."""
    print(f"\n\n>>> Ejecutando: test_unidad_insertar_producto")
    print(f" [SQL INSERT] Registrando de forma aislada en tabla 'producto': 'Camiseta Test Unitaria'...")

    resultado = True
    try:
        ProductoDAO.agregar_producto("Camiseta Test Unitaria", "M", "Verde", 29900.00, 1)
        print(f" [PERSISTENCIA] Inserción realizada en base de datos sin errores de sintaxis")
    except Exception as e:
        print(f" [ERROR PERSISTENCIA] Fallo en la inserción: {e}")
        resultado = False

    self.assertTrue(resultado, "La prueba unitaria de inserción de producto falló.")

```



```

IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help

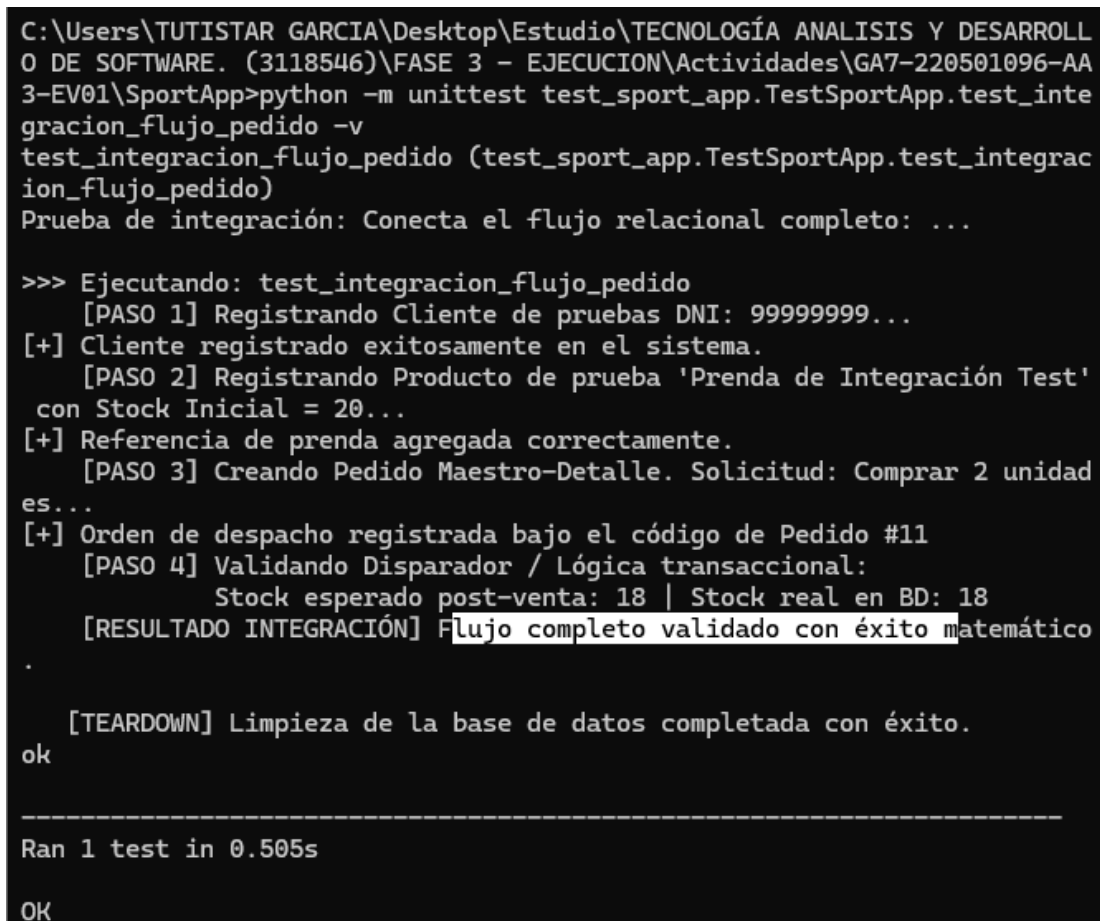
>>> Ejecutando: test_unidad_insertar_producto
[SQL INSERT] Registrando de forma aislada en tabla 'producto': 'Camiseta Test Unitaria'...
[+] Referencia de prenda agregada correctamente.
[PERSISTENCIA] Inserción realizada en base de datos sin errores de sintaxis.

[TEARDOWN] Limpieza de la base de datos completada con éxito.

```

C. Flujo Relacional Completo de Compras e Inventario (Integración):

```
python -m unittest test_sport_app.TestSportApp.test_integracion_flujo_pedido -v
```



```

C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE. (3118546)\FASE 3 - EJECUCION\Actividades\GA7-220501096-AA3-EV01\SportApp>python -m unittest test_sport_app.TestSportApp.test_integracion_flujo_pedido -v
test_integracion_flujo_pedido (test_sport_app.TestSportApp.test_integracion_flujo_pedido)
Prueba de integración: Conecta el flujo relacional completo: ...

>>> Ejecutando: test_integracion_flujo_pedido
[PASO 1] Registrando Cliente de pruebas DNI: 99999999...
[+] Cliente registrado exitosamente en el sistema.
[PASO 2] Registrando Producto de prueba 'Prenda de Integración Test' con Stock Inicial = 20...
[+] Referencia de prenda agregada correctamente.
[PASO 3] Creando Pedido Maestro-Detalle. Solicitud: Comprar 2 unidades...
[+] Orden de despacho registrada bajo el código de Pedido #11
[PASO 4] Validando Disparador / Lógica transaccional:
      Stock esperado post-venta: 18 | Stock real en BD: 18
[RESULTADO INTEGRACIÓN] Flujo completo validado con éxito matemático

[TEARDOWN] Limpieza de la base de datos completada con éxito.
ok

-----
Ran 1 test in 0.505s

OK

```

```
>>> Ejecutando: test_integracion_flujo_pedido
[PASO 1] Registrando Cliente de pruebas DNI: 99999999...
[+] Cliente registrado exitosamente en el sistema.
[PASO 2] Registrando Producto de prueba 'Prenda de Integración Test' con Stock Inicial = 20...
[+] Referencia de prenda agregada correctamente.
[PASO 3] Creando Pedido Maestro-Detalle. Solicitud: Comprar 2 unidades...
[+] Orden de despacho registrada bajo el código de Pedido #12
[PASO 4] Validando Disparador / Lógica transaccional:
    Stock esperado post-venta: 18 | Stock real en BD: 18
[RESULTADO INTEGRACIÓN] Flujo completo validado con éxito matemático.

[TEARDOWN] Limpieza de la base de datos completada con éxito.
```

```

test_sport_app.py - C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE (3118546)\FASE 3 - EJECU...
File Edit Format Run Options Window Help

# -----
# CATEGORÍA 3. PRUEBAS DE INTEGRACIÓN (FLUJO RELACIONAL COMPLETO)
# -----
def test_integracion_flujo_pedido(self):
    """
    Prueba de integración. Conecta el flujo relacional completo:
    Registra cliente -> Registra producto -> Genera pedido maestro-detalle -> Valida stock
    """
    print("\n\n>>> Ejecutando: test_integracion_flujo_pedido")
    dni_test = "99999999"
    desc_test = "Prenda de Integración Test"
    flujo_exitoso = False

    try:
        # 1. Registrar Cliente
        print("[ PASO 1] Registrando Cliente de pruebas DNI: {dni_test} .")
        ClienteDAO.insertar_cliente("Test", "Integracion", dni_test, "3000000", "Calle Falsa 123", "test@sena.edu.co")

        # 2. Registrar Producto con 20 unidades iniciales
        print("[ PASO 2] Registrando Producto de prueba '{desc_test}' con Stock inicial = 20 .")
        ProductoDAO.agregar_producto(desc_test, "L", "Azul", 45000.00, 20)

        # Recuperar IDs generados desde la base de datos
        conexion = obtener_conexion()
        cursor = conexion.cursor(dictionary=True)

        cursor.execute("SELECT id_clientes FROM clientes WHERE identificacion = %s", (dni_test,))
        fila_cliente = cursor.fetchone()

        cursor.execute("SELECT id_producto FROM producto WHERE descripcion = %s", (desc_test,))
        fila_producto = cursor.fetchone()

        if fila_cliente and fila_producto:
            cliente_id = fila_cliente['id_clientes']
            producto_id = fila_producto['id_producto']

            # 3. Registrar el Pedido usando "Pendiente" y comprando 2 unidades:
            print("[ PASO 3] Creando Pedido Maestro-Detalle. Solicitud: Comprar 2 unidades .")
            fecha_ahora = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            PedidoDAO.registrar_pedido_completo(fecha_ahora, "Pendiente", cliente_id, producto_id, 2)

            # Sincronizar el estado de la conexión actual para limpiar la cache de lectura de MySQL
            conexion.rollback()

            # Verificar el descuento automático de existencias (20 iniciales - 2 vendidas = 18 esperadas)
            cursor.execute("SELECT id_producto, stock FROM producto WHERE id_producto = %s", (producto_id,))
            fila_stock = cursor.fetchone()
            stock_resultante = fila_stock['stock'] if fila_stock else 20
            print("[ PASO 4] Validando Disparador / Lógica transaccional:")
            print("    Stock esperado post-venta: 18 | Stock real en BD: {stock_resultante}")

            # Validación del comportamiento de la base de datos
            self.assertEqual(stock_resultante, 18, f"Error de integración. El stock quedó en {stock_resultante} en vez de 18.")
            print("[ RESULTADO INTEGRACIÓN] Flujo completo validado con éxito matemático.")
            flujo_exitoso = True

        cursor.close()
        conexion.close()

    except Exception as e:
        print("[ -] Fallo en cadena de integración: {e}")
        flujo_exitoso = False

    self.assertTrue(flujo_exitoso, "El flujo completo de integración falló en la base de datos.")

```

D. Prueba de Conexión a la Base de Datos:

```
python -m unittest test_sport_app.TestSportApp.test_unidad_conexion_mysql -v
```

```

C:\WINDOWS\system32\cmd. x + v
C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y DESARROLLO DE SOFTWARE. (3118546)\FASE 3 - EJECUCION\Actividades\GA7-220501096-AA3-EV01\SportApp>python -m unittest test_sport_app.TestSportApp.test_unidad_conexion_mysql -v
test_unidad_conexion_mysql (test_sport_app.TestSportApp.test_unidad_conexion_mysql)
Verifica de forma aislada que el módulo conexion.py retorne un objeto activo. ...

>>> Ejecutando: test_unidad_conexion_mysql
[CONEXIÓN] Solicitando enlace con el servidor de MySQL en Tuluá...
[ESTADO MYSQL] ¿Base de datos conectada con éxito?: True

[TEARDOWN] Limpieza de la base de datos completada con éxito.
ok

-----
Ran 1 test in 0.234s

OK

```

```

test_sport_app.py - C:\Users\TUTISTAR GARCIA\Desktop\Estudio\TECNOLOGÍA ANALISIS Y D...
File Edit Format Run Options Window Help

def test_unidad_conexion_mysql(self):
    """Verifica de forma aislada que el módulo conexion.py retorne un objeto activo."""
    print(f"\n\n>>> Ejecutando: test_unidad_conexion_mysql")
    print(f" [CONEXIÓN] Solicitando enlace con el servidor de MySQL en Tuluá...")

    enlace = obtener_conexion()
    self.assertIsNotNone(enlace, "Error crítico: No hay comunicación con MySQL Server.")

    if enlace:
        estado = enlace.is_connected()
        print(f" [ESTADO MYSQL] ¿Base de datos conectada con éxito?: {estado}")
        self.assertTrue(estado)
        enlace.close()

```

```

IDLE Shell 3.12.7
File Edit Shell Debug Options Window Help

>>> Ejecutando: test_unidad_conexion_mysql
[CONEXIÓN] Solicitando enlace con el servidor de MySQL en Tuluá...
[ESTADO MYSQL] ¿Base de datos conectada con éxito?: True

[TEARDOWN] Limpieza de la base de datos completada con éxito.

```

Conclusión

Como conclusión de este proceso de testing, se destaca que las pruebas automatizadas constituyen un pilar indispensable para certificar que el software cumple estrictamente con las reglas de negocio e historias de usuario definidas. A través de la ejecución de comandos en consola y la validación de respuestas exitosas del sistema, se logró blindar la aplicación ante posibles fallos de integridad y vulnerabilidades comunes, tales como estructuras de correo incorrectas o ingresos anómalos de datos.